

Script: Velociraptor Artifact Merger

This is a well-designed utility that consolidates memory-forensics artifacts (pslist, netstat, threads, handles, DLLs, drivers, VAD, mutex, impersonation, procinfo) from a Velociraptor results folder into one clean, process-centric JSON file keyed by PID. It works by walking the input folder and matching filenames against ARTIFACT_MAP to organize files by category, auto-detecting each file's format (JSON array, JSONL, or CSV) for flexible loading, pulling PID/PPID from records using multiple field-name variants, and gracefully grouping everything into unified process objects — preserving unmatched records in a separate unattributed bucket rather than discarding them — before producing a neatly sorted merged_processes.json. Its strengths lie in impressively format-agnostic loading, tolerant key matching across naming conventions, careful preservation of every record, and clean, modular, easily extensible structure. Overall, it's a thoughtfully engineered, practical DFIR tool — readable, well-organized, and genuinely useful as a time-saving piece of an investigation pipeline.

Script: Evidence Builder (Full Version)

This is a comprehensive, well-architected engine that transforms extracted memory-forensic features into structured, explainable evidence — complete with severity scoring, MITRE ATT&CK mapping, correlation logic, and analyst-friendly summaries, ready for evidence.json.

It works by establishing a clean Evidence class and severity-scoring foundation, attaching curated MITRE ATT&CK technique IDs to findings, and using small well-considered utility functions to support a modular set of per-category rule engines (threads, DLLs, handles, tokens, netstat, mutex, procinfo, pslist, VAD, drivers) that each cover their forensic domain with sensible thresholds and clear “why it matters” explanations. Its standout feature is the correlation engine, which intelligently combines weak signals across categories — remote threads with RWX memory, LSASS access with debug privileges, foreign IPs with suspicious ports — to surface high-confidence composite findings like Process Injection, Credential Dumping, C2, and Kernel Rootkit detection; this feeds into a clean report-assembly and deduplication step, followed by a natural-language analyst summary and a well-structured final JSON output with risk, confidence, and score. Its strengths lie in excellent separation of concerns, thoughtful multi-signal correlation that mirrors real analyst reasoning, a consistent and extensible scoring model, valuable MITRE enrichment, and genuinely explainable output suited to both human analysts and downstream LLM consumption. Overall, it's a mature, thoughtfully engineered detection-and-reporting engine whose correlation layer reflects real threat-hunting logic, making it a strong, reusable foundation for the DFIR pipeline.

Script: Baseline Compare (baseline_compare.py)

This script serves as the fourth stage of the pipeline, comparing a baseline (known-good) snapshot against a current (incident) snapshot — both already processed by extract_features.py — to surface everything that requires two points in time to detect,

including handle-count spikes, new mutexes/DLLs/drivers, activated risk flags, PPID changes, and new or terminated processes. Its flow begins by resolving process identity via merged files (`build_pid_to_name/build_pid_to_ppid`), then matches processes by exact PID first before falling back to name-based pairing in sorted-PID order; it generically diffs every `*_features` block (`diff_leaf_tree`) for flag activations, count changes, and value changes, separately diffs named entities like mutexes and DLLs for brand-new items, flags count spikes using both a relative multiplier and an absolute minimum delta, computes weighted anomaly scores per process, diffs system-wide driver features, and finally writes `baseline_comparison.json` while printing a ranked top-N anomaly leaderboard to the console. Among its strengths are an excellent self-documenting docstring that clearly explains its purpose and matching strategy, a schema-agnostic design consistent with the upstream scripts, a sensible PID-exact-then-name-paired matching fallback, solid statistical spike detection that avoids noise on tiny counters, an extensible scoring model that appropriately weights malware-specific traits like known-malware mutexes and suspicious DLLs, and clean, well-labeled CLI output that's genuinely useful for analyst triage. Altogether, it is a well-documented, logically coherent stage that fills a real gap — temporal diffing — left open by the single-snapshot scripts, tying the whole pipeline together into a genuinely useful before/during incident comparison tool.

Script: Statistical Anomaly Scorer

This script provides a statistical, baseline-driven alternative to the rule-based evidence engines — instead of hardcoded thresholds, it computes a per-feature mean and standard deviation across all processes in a baseline snapshot, then scores each process in a current snapshot by summing the absolute z-scores of its numeric features against those baseline statistics. It works by flattening every process's numeric `*_features` values into dotted keys (`flatten_features`), computing per-key mean/std across the baseline population (`compute_stats`), and then for each current process calculating how many standard deviations each feature deviates from baseline, aggregating these into a single anomaly score alongside per-feature detail. Its strengths are simplicity, a genuinely different and complementary detection approach (population-based outlier detection rather than fixed thresholds), and transparent per-feature scoring that's easy to audit. However, a few gaps stand out: it compares each current process only against the global population baseline rather than its own matched baseline counterpart (no PID/name pairing like the earlier `baseline_compare` script), so it can't detect changes within a specific process over time — only whether a process looks statistically unusual relative to others in the baseline; it silently ignores non-numeric features entirely (booleans, strings, lists), missing signals like risk flags or new mutex/DLL names; and it has no protection against small baseline sample sizes, where mean/std computed from only 1–2 data points can be unreliable or misleadingly precise. Overall, it's a clean, lightweight complement to the rule-based and diff-based approaches — useful for surfacing statistically odd processes — but works best paired with the other scripts rather than as a standalone detector, since it currently lacks identity-aware baseline matching and non-numeric feature coverage.

Script: Feature Extractor (extract_features.py)

This is the second stage of the pipeline, sitting between the artifact merger and the evidence builder — it reads merged_processes.json and computes the actual per-plugin detection features (mutex entropy, RWX execution, ASLR/DEP status, PEB mismatches, and roughly 90 keys per category) for every process, producing features.json. It works by running dedicated extractors for each artifact category (mutex, threads, handles, impersonation, DLLs, procinfo, netstat, pslist, VAD, drivers), each pulling raw fields through a resilient get_field lookup across multiple candidate names, then deriving boolean/count-based indicators like reflective DLL loading, LSASS handle access, suspicious ports, orphaned processes, and parent-child mismatches; a shared ctx (context) object supplies cross-process data like mutex frequency counts and process-tree depth, while drivers are handled once globally since they're system-wide rather than per-process. Its strengths are considerable: an unusually thoughtful and honest docstring that explicitly documents what can't be computed from a single snapshot (handle-count spikes, GeolIP grouping, malware reference lists) rather than silently omitting those keys; the same schema-agnostic, field-name-tolerant philosophy as the upstream merger; sensible use of regex-based heuristics (GUID-shaped mutex names, Base64/URL detection in command lines, homoglyph spoofing checks); and a genuinely clever recursive compute_depth_map with cycle protection for building process-tree depth. Overall, it's a mature, well-documented feature-engineering stage that faithfully bridges raw artifact data into the structured signal the downstream evidence and correlation engines depend on, while being transparent about its own known limitations rather than overstating what a single snapshot can reveal.

The Combined Threat Intelligence Pipeline integrates three sequential modules—Sysmon CSV IoC extraction, PCAP-based IoC extraction with domain validation, and multi-source enrichment—to systematically transform raw system and network data into actionable threat intelligence, where the logic begins by parsing Sysmon logs to extract indicators such as IP addresses, domains, file hashes, and process names using structured field access and regex filtering, followed by analyzing PCAP files to capture network-level IoCs like source/destination IPs and DNS queries, and then validating suspicious domains using VirusTotal to assess malicious or suspicious activity levels; after collecting these IoCs, the pipeline performs filtering and enrichment by querying multiple threat intelligence sources (VirusTotal, MalwareBazaar, and optionally AbuseIPDB) to attach contextual metadata such as malware family, abuse confidence scores, and detection statistics, thereby creating a consolidated and enriched dataset (threat_intel_output.json), and the key gain from this logic is that it enables comprehensive visibility across both host-based and network-based activities, improves detection reliability by correlating multiple independent intelligence sources, reduces noise through cleaning and filtering of raw IoCs, and produces a structured, enriched output that can be directly used by downstream modules for scoring, reasoning, and incident response, ultimately allowing security analysts to detect threats more accurately, understand their context, and respond more efficiently.

The provided **Combined Host Assessment Pipeline** implements a two-stage cybersecurity scoring and decision system that evaluates whether a host is compromised by combining threat intelligence with behavioral anomaly detection; in the first module, the logic processes enriched IOC data from multiple intelligence sources such as VirusTotal, AbuseIPDB, and MalwareBazaar, assigning weighted scores based on severity thresholds (e.g., high malicious detections, abuse confidence, or confirmed malware family), aggregating these scores across all indicators to produce a total risk score, and then classifying the host into CLEAN, SUSPICIOUS, or COMPROMISED categories based on predefined score ranges, while also preserving per-IOC scoring details for transparency; in the second module, the logic extends this by integrating anomaly detection outputs, where raw anomaly scores (z-scores) are normalized to a 0–100 scale by comparing the most suspicious process against the maximum observed score, and then fused with the threat intelligence score using a weighted model (60% threat intelligence, 40% anomaly behavior) to compute a final composite score that reflects both known malicious indicators and unusual system behavior, followed by a final verdict using similar thresholding; the key gain from this combined logic is that it significantly improves detection accuracy and contextual awareness by not relying solely on static threat intelligence (which may miss novel attacks) or anomaly detection alone (which may produce false positives), but instead fusing both to provide a more balanced, explainable, and reliable host compromise assessment, enabling security analysts to make faster, data-driven decisions with reduced false positives and better visibility into both known and unknown threats

The LLM Reasoning Engine (Module 13) acts as the final analytical layer in the pipeline, where it integrates multiple upstream inputs such as evidence data, baseline snapshots, threat intelligence, and host compromise scores to generate an analyst-level report; its core logic involves first constructing a severity-based timeline by extracting events from processes and ordering them using timestamps or severity when timestamps are missing, then reducing complex forensic data into compact evidence graphs to highlight only high-risk activities, followed by comparing incident data with baseline data to distinguish genuinely new or anomalous behavior from normal system activity, and correlating indicators of compromise with threat intelligence to validate maliciousness; after preprocessing, it applies structured LLM reasoning that includes mapping activities to the attack kill chain, establishing causal relationships between processes, validating anomalies using baseline comparison, cross-checking with threat intelligence, and verifying conclusions against computed risk scores while also considering possible benign explanations, ultimately producing a standardized JSON output containing attack narrative, explanation of what happened and why, severity assessment, and actionable recommendations; the key gain from this approach is that it transforms large volumes of raw forensic data into clear, human-readable cyber threat intelligence, significantly reduces analyst effort, improves detection accuracy by minimizing false positives through contextual reasoning, provides a complete understanding of attacker behavior, and enables faster and more informed incident response decisions.